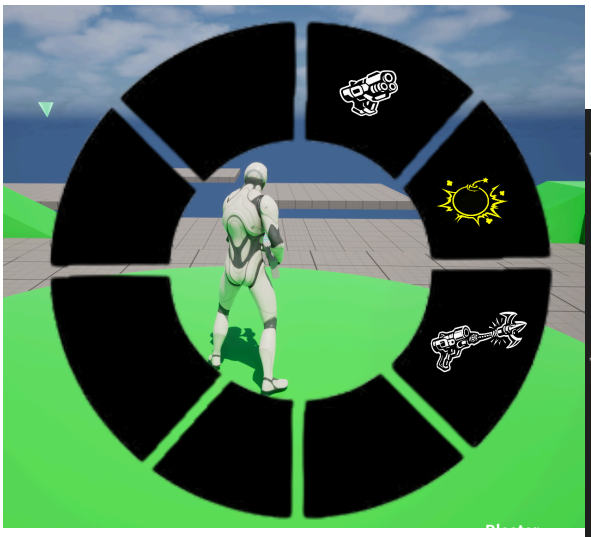
Weapon Wheel Code Analysis

Repo: https://eae-git.eng.utah.edu/u1537288/squirrelactionplatformer

Did you start from scratch? Or did you revisit a previous game project (if so, which one)? Explain/Discuss your choice.

This project was built entirely from scratch. For each of my projects, I had a design, engineering, and reference goal in mind. For this project, the reference was to make a weapon system similar to Ratchet and Clank. I wanted to do this because those games were core childhood memories. In previous projects, I chose to do a dark souls inspired where the main focus was not just the assignments, but level, environment, and encounter design. There are a multitude of projects that I wanted to tackle, the main one being a Ratchet and Clank-like game, so building off this project after graduating is almost certain. Not to mention potentially finding a job with Insomniac, building a game like this will help me stand out.

The main skills I wanted to refine were the type object and how it can be used in tandem with other patterns, but also to make it more organized, as my last one went a little broad. With this more refined approach, I was able to capture the weapon and gadget classifications of tools for the Ratchet and Clank games, and do it from one data asset. This allows me to switch weapon models, projectile classes, and a multitude of weapon-related fields. The main goal was to create the weapon wheel, which the game is well known for, and I have accomplished this according to my initial goals.



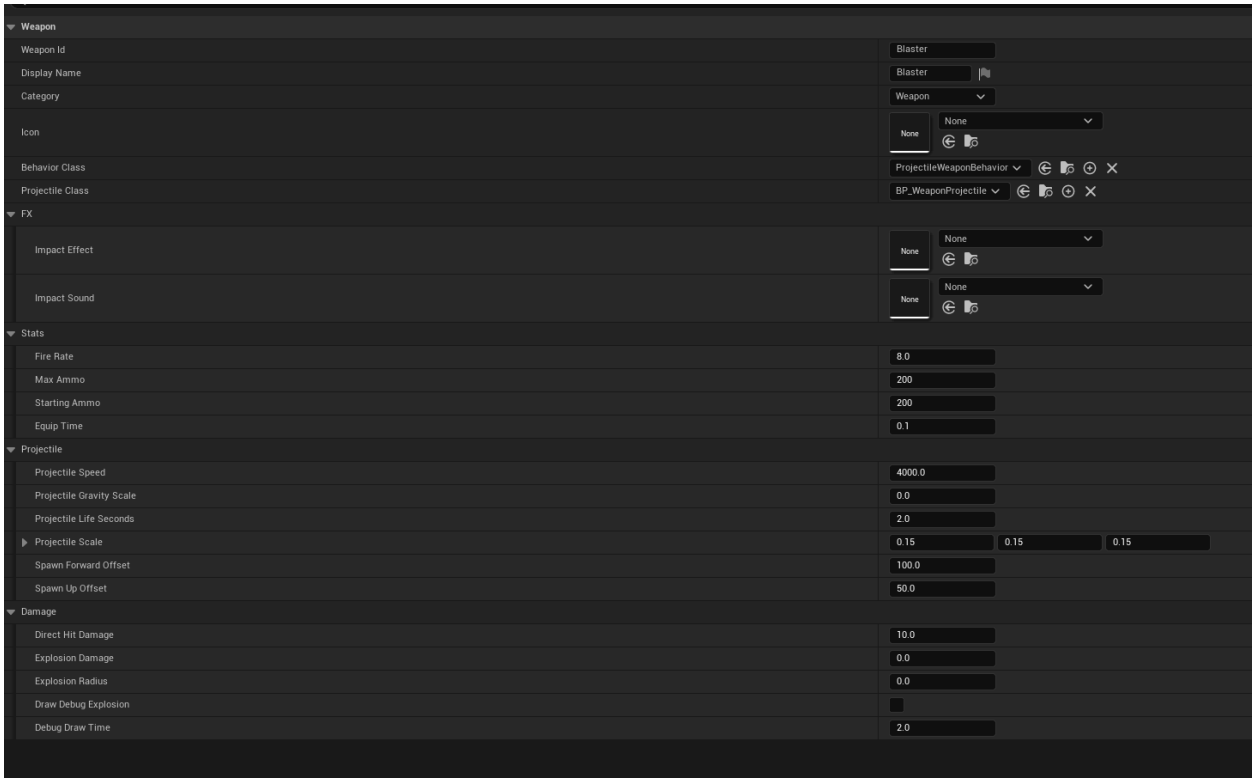
```cpp
#include "CoreMinimal.h"
#include "UObject/Object.h"
#include "WeaponBehaviorBase.generated.h"

class UWeaponInstance;

UCLASS(Abstract, Blueprintable, EditInlineNew, DefaultToInstanced)
0 derived Blueprint classes
class SQUIRRELPLATFORMER_API UWeaponBehaviorBase : public UObject
{
    GENERATED_BODY()

public:
    virtual bool CanStartUse(UWeaponInstance* WeaponInstance) const;
    virtual void StartUse(UWeaponInstance* WeaponInstance);
    virtual void StopUse(UWeaponInstance* WeaponInstance);

    virtual void OnEquip(UWeaponInstance* WeaponInstance);
    virtual void OnUnequip(UWeaponInstance* WeaponInstance);
};
```



Discuss your code implementation

The main focus of my implementation was building a modular, data-driven system that supports both weapons and gadgets. I used the Type Object pattern through UWeaponDataAsset to define weapon data separately from code, allowing easy creation and tuning of different weapons. I also used a Strategy pattern with UWeaponBehaviorBase, where each weapon or gadget (blaster, bomb launcher, swingshot) defines its own behavior while the player delegates execution to the current behavior.

I implemented a simplified State pattern using enums to manage movement and action states. The weapon wheel uses a radial selection system based on mouse position for quick switching. For the swingshot, I switched from line tracing to a screen-space targeting system, which highlights anchors based on their position on screen to improve usability. Something I learned from the previous assignment and for all future projects is that the state pattern makes maintaining animation blueprints 10x easier, and adding new states is so easy and fast.

```
UENUM(BlueprintType)
enum class EMovementState : uint8
{
    Idle    UMETA(DisplayName = "Idle"),
    Walk    UMETA(DisplayName = "Walk"),
    Run     UMETA(DisplayName = "Run"),
    Crouch  UMETA(DisplayName = "Crouch"),
    Jump    UMETA(DisplayName = "Jump")
};

UENUM(BlueprintType)
enum class EActionState : uint8
{
    Neutral     UMETA(DisplayName = "Neutral"),
    Firing      UMETA(DisplayName = "Firing"),
    GadgetUse   UMETA(DisplayName = "GadgetUse"),
    WeaponWheel UMETA(DisplayName = "WeaponWheel")
};
```

What are one (or two) lessons you learned while making this game?

One key lesson was that building reusable systems is more valuable than isolated features. Creating a flexible weapon system made it much easier to add new mechanics like the swingshot. Starting from scratch allowed me to take what I learned and tighten it up, and make it organized and modular. Now, designers can take my data asset and easily create different types of weapons just based on what is currently a part of the type object, with additional fields easily expandable upon request.

Another lesson was that gamefeel is more important than technical precision. The original swingshot required precise aiming and felt frustrating, but switching to a more forgiving, highlight-based system made it much more intuitive and enjoyable. I looked directly at Ratchet and Clank to see how it worked there. It is more of a target system based on whether the player had the grapple point within a certain range and radius of the camera.

Catch-All

One of the most interesting parts of this project was integrating the swingshot gadget into the same system as weapons. Although traversal and combat are very different mechanically, treating them as interchangeable behaviors through the weapon system created a clean and unified architecture. A challenge encountered was tuning the targeting system. Early versions required too much precision, which made the mechanic frustrating. Iterating on this system to prioritize player intention (highlight-based targeting) was a key improvement. I would (and still plan) to have done more, but am currently crunching on my Thesis Project.

Time Spent

I spent around 20 hours, mainly getting the state pattern set up again, as I was looking over my last project to save time, but I also questioned a lot, as it was my first time setting that up. The whole weapon behavior and type object took the most time, because I wanted to get it into a state that allowed me to create different weapons from one data asset. There are still more fields I would need to add, but thinking about the behaviors of different weapons from Ratchet and Clank, there is potential to make different-feeling guns from what I have now. Lastly, I was setting up the Radial weapon wheel and adding a bit of polish to hide unused slots. Currently, I have some values hardcoded as each weapon goes to a specific slot, but I plan to add an interact/pickup system that will put them into the slot in clockwise order based on the order you pick them up in. Also, like in the Ratchet and Clank games, edit the slot manual to the designated layout/loadout order.